

Reporte: Actividad 3.1

Alumnos:

- Juan Carlos Malanco Rodriguez
- Isabel Kyra Romero Borunda
- Jocelyn Flores Gutiérrez

Maestro: Adalberto Emmanuel Rojas Perea

Materia: Desarrollo de Aplicaciones Móviles

Fecha: 04/09/2026

Descripción

Aplicar los fundamentos de Jetpack Compose creando una miniaplicación interactiva que integre *composables* personalizados, *layouts* adaptativos, manejo de estados y personalización de tema, consolidando los conceptos clave del desarrollo de interfaces modernas en Android.

Instrucciones

1. Define el contexto:
 - Elige una temática sencilla para tu miniapp (por ejemplo, lista de tareas, contador de clics, catálogo de productos).
 - Escribe en una línea el objetivo de la aplicación (por ejemplo, "Permitir al usuario agregar y marcar tareas como completadas").
2. Crea la estructura básica en Compose:
 - Define al menos dos composables propios (por ejemplo, TaskItem, TaskInput).
 - Utiliza layouts como Column, Row o LazyColumn para organizar los elementos en pantalla.
3. Integra componentes interactivos y manejo de estado:
 - Implementa al menos un campo de texto (TextField) y un botón (Button) que modifiquen el estado de la aplicación (por ejemplo, agregar elementos a una lista).
 - Usa remember y mutableStateOf para almacenar y actualizar el estado.
4. Personaliza el tema y adapta la interfaz:
 - Aplica un esquema de colores propio modificando el MaterialTheme.
 - Asegúrate de que los elementos sean legibles en modo claro y oscuro.
 - Si es posible, utiliza Modifier para ajustar el diseño a diferentes tamaños de pantalla.
5. Prueba y documenta:
 - Ejecuta la aplicación en el emulador o dispositivo físico.
 - Toma capturas de pantalla y anota brevemente qué aprendiste y qué mejorarías.

1. Definir el contexto:

- **Temática:** Lista de tareas
- **Objetivo:** Permitir al usuario marcar y desmarcar las tareas como completadas y agregar nuevas.

2. Crear la estructura básica en Compose:

- Definir al menos dos compostables propios (Ejem. TaskItem, TaskInput)

```
2 Usages
@Composable
private fun TaskRow(
    task: Task,
    onToggle: () -> Unit
> ) {...}
```

```
2 Usages
@Composable
private fun SectionLabel(text: String) {...}
```

- Utilizar Layouts como Column, Row o LazyColumn para organizar los elementos en pantalla

```
// ----- BOX RAÍZ -----
Box(
    modifier = Modifier
        .fillMaxSize()
        .background( color = ScreenBg)
) {

    // ----- COLUMN PRINCIPAL (contenido vertical) -----
    Column(
        modifier = Modifier.fillMaxSize()
    ) {
```

3. Integra componentes interactivos y manejo de estado:

- Implementa al menos un campo de texto (TextField) y un botón (Button) que modifiquen el estado de la aplicación (por ejemplo, agregar elementos a una lista).

```
// ----- FAB flotante (posicionado dentro del Box raíz) -----
FloatingActionButton(
    onClick = { /* acción de agregar tarea */ },
    containerColor = HeaderRed,
    contentColor = Color.White,
    shape = CircleShape,
    modifier = Modifier
        .align(Alignment.BottomEnd)
        .padding(all = 24.dp)
) {
    Icon(imageVector = Icons.Default.Add, contentDescription = "Agregar tarea")
}
```

- Usa remember y mutableStateOf para almacenar y actualizar el estado.

```
@Composable
fun MyTasksScreen() {

    var selectedFilter by remember { mutableStateOf(value = "All Tasks") }

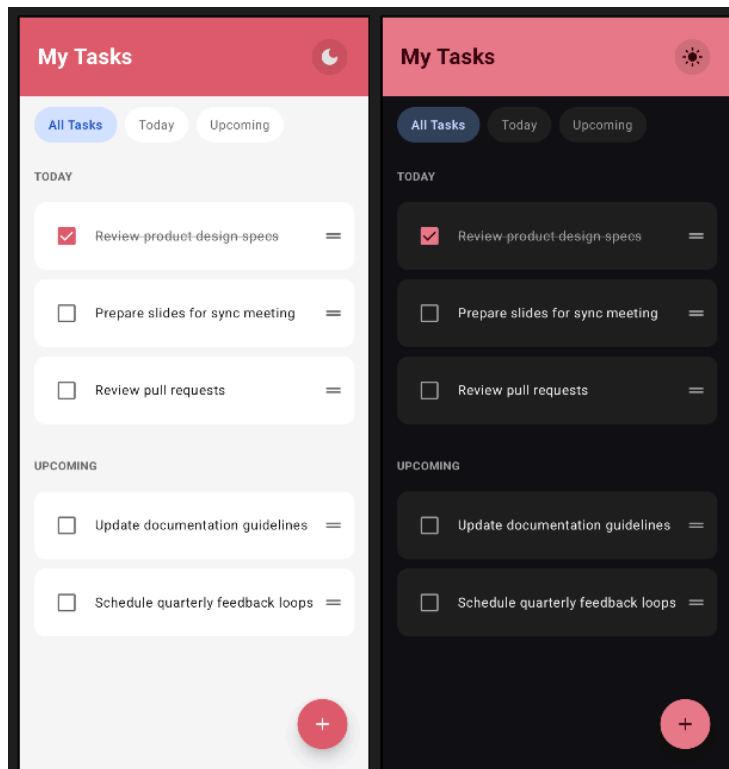
    val todayTasks = remember {
        mutableStateListOf(
            Task(title = "Review product design specs", done = true),
            Task(title = "Prepare slides for sync meeting"),
            Task(title = "Review pull requests")
        )
    }
}
```

4. Personaliza el tema y adapta la interfaz:

- Aplica un esquema de colores propio modificando el MaterialTheme.

<pre>private val LightColors = lightColorScheme(primary = Color(0xFFE05B6B), onPrimary = Color.White, background = Color(0xFFFFF5F5), surface = Color.White, onSurface = Color(0xFF1A1A1A), onSurfaceVariant = Color(0xFF6B6B6B), secondaryContainer = Color(0xFFD6E4FF), onSecondaryContainer = Color(0xFF245C9))</pre>	<pre>1 Usage private val DarkColors = darkColorScheme(primary = Color(0xFFE87C89), onPrimary = Color(0xFF3A0410), background = Color(0xFF121214), surface = Color(0xFF1E1E20), onSurface = Color(0xFFE0E0E0), onSurfaceVariant = Color(0xFFA0A0A0), secondaryContainer = Color(0xFF33415C), onSecondaryContainer = Color(0xFFC6D8FF))</pre>
--	---

- Asegúrate de que los elementos sean legibles en modo claro y oscuro.



- Si es posible, utiliza Modifier para ajustar el diseño a diferentes tamaños de pantalla.

